ARCHITECTURE AND DECISION RECORD

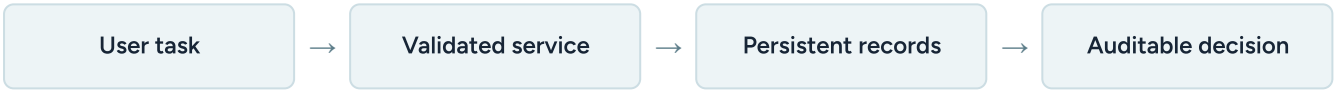
A document request and controlled records-release portal

Patricia Ramos · Database architecture and secure records reviewer

PostgreSQL | Laravel | Authorization | Audit design

Problem and contribution

This original design case study explores how to track document requests and authorized release while preserving version provenance and restricting access to sensitive files. I authored the requirements breakdown, architecture boundaries, data relationships, failure scenarios, and review rationale presented here. The design is a fictional presentation example, with no claim of a production deployment or measured client result.



Architecture decisions and tradeoffs

- Keep sensitive storage outside the public web root and authorize each download at request time.
- Never replace a released file in place; a corrected document gets a new version and authorization decision.
- Make retention an explicit reviewed policy; deletion should respect legal and institutional requirements before deployment.

Core records

Record	Responsibility
document_types	A type defines required fields and the applicable approved retention rule.
requests	A requester has many requests; transitions retain their actor and reason.
requirement_checks	Checks record missing or accepted requirements without overwriting prior decisions.
record_versions	A new version references the earlier record and keeps provenance.

The service layer owns authorization and business constraints. Interface validation improves the experience but cannot replace server checks. Event records preserve attributable decisions and their timing without logging unnecessary private content.

VERIFICATION AND REVIEW PRACTICE

Failure cases, evidence, and limitations

Proposed verification matrix

Scenario	Method	Expected behavior
Direct object access	Change a request, version, or authorization identifier while signed in as another requester.	The server denies access even when the URL is known.
Expired authorization	Attempt a download after expiry or after a version is superseded.	The file is not released and the requester receives an appropriate next step.
Correction and reissue	Prepare a correction after an earlier document has already been released.	The history shows both versions, the correction reason, and the new approval.
Workflow comprehension	Ask requesters and processors to find missing requirements and next actions.	Record task success, errors, and clarity without assuming that a completed screen proves correct release.

Technical review

Protect downloads with server-side ownership checks and store files outside the public web root. A correction should create a new document version and approval record rather than silently replacing a released file.

How to evaluate the design

Measure unauthorized access rejection, version-to-release traceability, task completion, and time spent resolving missing requirements. Audit checks should verify attributable events without copying document bodies into logs. No actual student records are used in this proposal.

The matrix specifies intended tests, not completed test results. A reviewer should capture the environment, initial records, inputs, expected state, observed state, and any unresolved defect before interpreting the outcome. Repeated runs should use separate unique request keys where relevant and preserve failures for analysis.

Scope limitations

No legally certified digital signatures, identity-document collection, payment processing, real academic transcript claims, or production retention guarantees.

Review whether every released file can be traced to an approved version and recipient, including after a correction or access-policy change.

> **Authorship and reuse**
>
> Prepared by Patricia Ramos as an original presentation work sample. All narrative, schemas, and test scenarios are original illustrative material. There are no external client documents, employment certificates, real personal records, or fabricated citations in this case study.